An Introduction to Reinforcement Learning

Caleb Lee, Catherine Cao, Arshia Mago, Ashlee King, Christina Fan[a]

[a] *University of Waterloo, Department of Psychology, 200 University Avenue West, Waterloo, N2L 3G1*

Abstract

This is the abstract. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Vestibulum augue turpis, dictum non malesuada a, volutpat eget velit. Nam placerat turpis purus, eu tristique ex tincidunt et. Mauris sed augue eget turpis ultrices tincidunt. Sed et mi in leo porta egestas. Aliquam non laoreet velit. Nunc quis ex vitae eros aliquet auctor nec ac libero. Duis laoreet sapien eu mi luctus, in bibendum leo molestie. Sed hendrerit diam diam, ac dapibus nisl volutpat vitae. Aliquam bibendum varius libero, eu efficitur justo rutrum at. Sed at tempus elit.

Keywords: Q-learning, Reinforcement learning, State, Agent, Action, TD learning

1. Introduction

1.1. History

- Ashlee History

1.2. Importance and Applications

- Arshia + some Catherine applications stuff

1.2.1. Applications

Reinforcement learning has been successfully applied to a wide range of domains. Sutton and Barto (2020) explain that an agent learns to interact with an environment through trial and error, in order to achieve long-term goals under uncertainty. The agent's actions influence both immediate outcomes and future states, requiring decisions that account for delayed consequences and involve planning. At the same time, because outcomes are uncertain, the agent must continuously monitor the environment and adjust its behaviour. Over time, the agent improves its performance through experience, updating its decision-making strategies (Sutton & Barto, 2020).

*Corresponding author

Robotics is one of the most widely studied domains of reinforcement learning, with applications in manufacturing, logistics, healthcare, and space (Li, 2019). Reinforcement learning enables robots to learn complex sequential behaviors through interaction with their environment, making it well-suited for tasks that involve uncertainty and dynamic conditions (Li, 2019). According to Kober et al. (2013), representations, prior knowledge, and approximate models are fundamental components in robotics. Representations refer to state-action discretization, function approximation, and pre-structured policies, and they aim to make complex reinforcement learning problems tractable. Prior knowledge is crucial in guiding the learning process and improving efficiency, including initial policies, demonstration, task structuring, and constraints on actions or policies. Approximate models also improve learning efficiency by enabling simulation and prediction of environment dynamics. In robotics, this allows for techniques such as mental rehearsal, where agents are trained in simulated environments to reduce real-world interaction, and learned forward models (Kober et al., 2013). However, Li (2019) states that discrepancies between simulation and the real world, referred to as the sim-to-real gap, might be a challenge. It could be addressed by improving the simulator or designing policies such as randomization in dynamics, observations, and environmental conditions (Li, 2019). In addition, Akalin and Loutfi (2021) explain that the development of social robots has introduced new forms of reward mechanisms. In interactive reinforcement learning, robots learn from human feedback, which may be explicit (e.g., ratings or labels) or implicit (e.g., gestures, speech, or emotional cues).

Reinforcement learning enables self-driving cars to learn optimal driving policies by rewarding desired driving behaviors and penalizing errors, allowing agents to learn through trial and error and ultimately reach the destination (Sivamayil et al., 2023). According to Badue et al. (2021), the self-driving system is typically divided into the perception system and the decision-making system. The perception system is responsible for interpreting data from sensors such as cameras, Light Detection and Ranging (LIDAR), Radio Detection and Ranging (RADAR) and the Global Positioning System (GPS) to form a representation of the environment. It involves road mapping, vehicle localization, and the detection of static and dynamic obstacles. The decision-making system uses this information to determine appropriate actions for the vehicle. It considers the current state of the vehicle, the environment, and constraints such as traffic regulations, and performs route planning, path planning, obstacle avoidance and vehicle control (Badue et al., 2021).

Recommender systems aim to predict user preferences for products and services, to reduce information overload (Li, 2019). They personalize user experience by understanding and analyzing user behaviour, preferences, and interaction patterns, such as satisfaction, interests, and engagement (Li, 2019). Afsar et al. (2023) state that in traditional recommendation systems, collaborative filtering methods use patterns of user interactions to identify similarities between users or items, whereas content-based filtering relies on item features and user profiles to match users with relevant content. Modeling the recommendation

systems as a sequential decision-making process can better reflect the user-system interaction. It is modeled as a Markov decision process (MDP) and uses reinforcement learning algorithms. Reinforcement learning is suited for recommendation systems because it constantly updates its policy based on rewards and accounts for long-term rewards. For recommendation systems, it learns from the user-system interaction, receives feedback as rewards, and maximizes long-term user engagement, satisfaction, and revenue (Afsar et al., 2023).

2. Mathematical Theory

2.1. Markov Decision Process

The Markov Decision Process (MDP) forms the mathematical foundation of reinforcement learning algorithms. It provides a framework for modeling sequential decision-making problems in stochastic environments, where an agent's actions influence both immediate rewards and subsequent states. These systems satisfy Markov property, meaning that the state transition and reward depend only on the current state and action, regardless of the prior process.

A MDP is defined by states, actions and rewards, which together describe the interaction between an agent and its environment. The set of all possible states is denoted by $\mathcal{S}$, where each state S_t represents specific coordinates or condition of the environment at time step t. The agent observes the current state S_t and selects an action A_t, where $\mathcal{A}(s)$ denotes the set of available actions in that state.

After taking action A_t, the agent transitions to a new state S_{t+1}. The state transition probability is the likelihood of transitioning to a subsequent state s' given the current state-action pair (s, a):

$$P(S' \mid s, a) = \Pr(S_{t+1} = S' \mid S_t = s, A_t = a)$$

Simultaneously, the agent receives an expected immediate reward $R(s, a)$, which is defined as the expected reward received after taking action A in state S:

$$R(s, a) = \mathbb{E}\left[R_{t+1} \mid S_t = s, A_t = a\right]$$

2.2. Policy and Value Function

A policy, denoted as π, specifies agent's behaviour by defining how actions are selected in each state, $\pi(a \mid s) = \Pr(A_t = a \mid S_t = s)$ represents the probability that the agent selects action a when in state s.The state-value function, $v_\pi(s)$, represents the expected cumulative reward received when the agent starts in state s and follows policy π. It is defined as:

$$v_\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,\middle|\, S_t = s \right]$$

where $\gamma \in [0, 1]$ is the discount factor, which balances immediate reward and the long-term reward. A smaller gives more weight to immediate rewards, while a closer to 1 gives more weight to long-term rewards.

To evaluate the value of specific actions taken in a state, we use the action-value function (or Q-function), $q_\pi(s, a)$. This function measures the expected cumulative reward when the agent starts in state s, takes action a, and follows policy π:

$$q_\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,\middle|\, S_t = s, A_t = a \right]$$

2.3. Bellman Equation

The Bellman equation for v_π shows a recursive relationship between the value of a state and the values of subsequent states. It describes the value of a state as the sum of the expected immediate reward and the discounted value of subsequent states. For any policy π, the state-value function satisfies:

$$v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_\pi(s') \right]$$

The Bellman optimality equation can be expressed without reference to a specific policy π, instead selecting the best possible action. The Bellman optimality equation for the optimal state-value function is:

$$v^*(s) = \max_{a \in \mathcal{A}(s)} \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v^*(s') \right]$$

It states that the optimal value of a state is the maximum expected total reward achievable by choosing the best action, and assuming optimal behaviour in subsequent states. The Bellman optimality equation for the optimal action-value function states the optimal value of a action-state pair:

$$q^*(s, a) = \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma \max_{a'} q^*(s', a') \right]$$

It represents the maximum expected total reward obtained by taking action a in state s, followed by optimal behaviour thereafter. It forms the foundation for Q-learning algorithm.

2.4. Q-Learning

A model of the environment refers to any mechanism that allows the agent to predict the outcome of its actions. Reinforcement learning methods can be categorized into model-based and model-free methods, depending on whether the agent has access to a model of the environment. Model-based agents utilize planning, which is a computational process that uses a model to improve a policy. The agent uses its model to generate simulated experience and evaluate future

trajectories. In contrast, model-free agents learn directly from real experience by interacting with the environment, executing actions and observing outcomes.

Q-learning is categorized as a model-free, off-policy algorithm. The primary goal of off-policy learning is to evaluate and optimize a target policy, which is the strategy intended to become optimal. Simultaneously, the agent follows a distinct, more exploratory behaviour policy to interact with the environment. The ϵ-greedy policy is used in Q-learning to select an action, which balances the trade-off between exploration and exploitation. With probability of ϵ, the agent explores by selecting a random action (or the non-greedy action) to improve its estimates of the optimal values. With probability $1 - \epsilon$, the agent exploits its current knowledge by choosing the greedy action, which is the action with the highest current estimated value.

Q-learning is a Temporal-Difference (TD) control algorithm that directly approximates the optimal action-value function. TD error, δ_t, represents the discrepancy between the current estimate of Q-value and the updated estimate of future value from the next state:

$$\delta_t = R_{t+1} + \gamma Q(S_{t+1}, a') - Q(S_t, A_t)$$

The Q-learning update rule utilizes this error to refine its estimates:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

where $\alpha \in [0, 1]$ is the learning rate (or step-size parameter). The Q-value $Q(S_t, A_t)$ is the current estimate of the value of taking action A in state S. It is updated by incorporating the immediate reward and the discounted value of the best possible action in the subsequent state. This update assumes that the agent follows the optimal policy from the next state onward. A Q-table is a tabular representation of the Q-values, it presents the state-action value estimates. This table is updated iteratively using the Q-learning update rule, until the values converge to the optimal action-value function $Q^*(s, a)$. As convergence is achieved, the optimal policy π^* can be obtained by selecting the action that maximizes the value for any given state: $\pi^*(s) = \arg\max_a Q^*(s, a)$.

3. Methods

- Christina and Caleb ## Implementation We built and implemented a Q-learning algorithm within a maze environment. The environment was represented as a 6×6 grid constructed using a NumPy array, where integer values encoded state types and associated rewards. Specifically, cells were defined as follows: 0 represented empty traversable space, 1 represented impassable walls, 2 represented fatal states ("holes") that returned the agent to the starting position, and 3 represented the goal state associated with a positive reward and episode termination.

A Q-table was initialized with zeros to represent the expected value of each state–action pair. The learning rate ( ), future discount factor ( ), and exploration rate ( ) were predefined and held constant throughout training. The agent's action space consisted of four discrete movements (up, down, left, right).

At each step, the agent selected an action using the Epsilon-Greedy strategy. State transitions were determined by applying the selected action to the agent's current position, with resulting coordinates mapped onto the grid to determine the subsequent state and corresponding reward.

Q-values were iteratively updated using the temporal difference learning rule based on the observed reward and the maximum expected future reward from the subsequent state. Training was conducted over a fixed number of episodes, with each episode beginning at the starting state and terminating upon reaching the goal or a terminal penalty state.

3.1. Neuropsychiatry Extension

- Caleb Part
- Q learning in computational psychiatry
- Example: D1/D2 receptors

4. Results

4.1. Model

After running 1000 episodes, we present evidence of the agent's learning process. Figure 1 illustrates changes in the number of steps the agent takes to complete one episode over the course of 500 episodes.

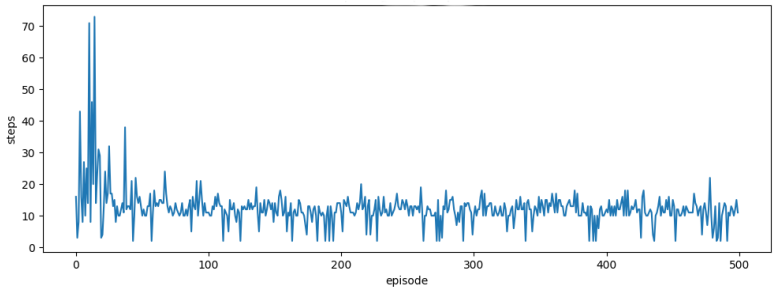

Figure 1: Changes in Steps to Complete One Episode

Figure 2 presents changes in the TD learning error (delta) over time across 6000 episodes.

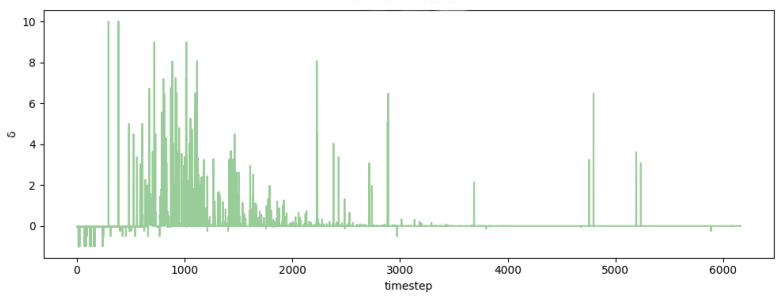

Figure 2: TD Error Over Time

Figure 3 is a Q-value heatmap showing the Q values of the best action to take in a given state (i.e. state-action pair that renders the maximal Q-value), with greater Q-values of darker color.

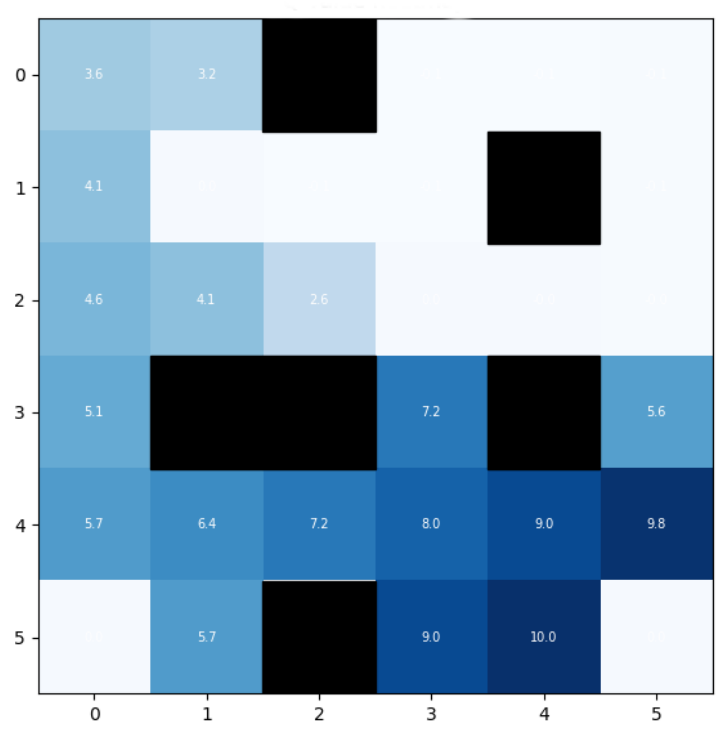

Figure 3: Q-Value Heatmap

Figure 4 illustrates the optimal actions to take in a given state, with visualizations of actions as arrows in the maze environment. This represents the optimal

policy the agent is learning.

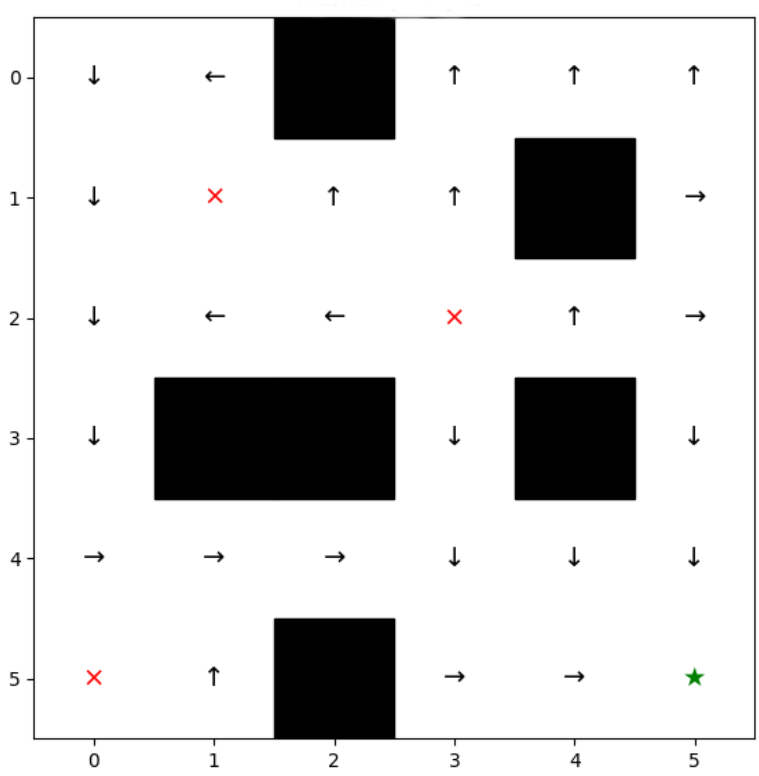

Figure 4: Optimal Learned Policy

4.2. Neuropsychiatry Extension

4.2.1. Dopamine as a Prediction Error Signal

- Schultz et al. (1997)
- DA neurons fire in response to positive prediction error/pause for neg
- This maps onto TD error: biological motivation for using RL as a model for brain function
- D1/D2 receptor mechanisms
- Frank et al. (2004): DA medications in Parkinson's reverse Go/NoGo learning biases/comparison with predictions made with RL models

The application of reinforcement learning algorithms in the study of the brain emerged with advancements in electrophysiological recording techniques in the 1990s. These techniques included single-unit recordings of individual neurons in alert subjects performing behavioural tasks. This allowed the investigation of firing patterns in dopaminergic (DA) neurons in the midbrain, which had been

proposed to be responsible for reward-driven learning. Single-unit recordings of DA neurons in the ventral tegmental area (VTA) and substantia nigra of monkeys performing classical conditioning tasks revealed a unique firing pattern: early in the trials, DA neurons fired phasically at reward delivery; after learning, they fired at the appearance of the cue that predicted reward, not at the reward itself; and when a predicted reward failed to arrive, their firing rate dropped below baseline at exactly the moment the reward would have occurred (Schultz et al., 1997). This suggested that dopaminergic neurons do not simply encode the occurrence of reward, but rather the descrepancy between the predicted and actual timing and magnitude of reward outcomes (Schultz et al., 1997). This discovery motivated a connection to reinforcement learning algorithms, specifically to the temporal difference (TD) error.

Schultz, Dayan, and Montague (1997) interpreted these findings with the TD algorithm, showing that the pattern of DA neuron firing matched the prediction error δ_t (see **?@eq-td-error**). When the TD model was applied to the same conditioning task that the monkeys performed, the prediction error it generated corresponded precisely to the three observed firing patterns: a positive prediction error when unexpected reward occurred, no error when reward was received as predicted, and a negative prediction error when an expected reward was absent. This established dopamine as a biological implementation of the TD error signal, suggesting that the brain computes something equivalent to TD learning through phasic DA neuron firing activity.

While Schultz et al. (1997) established that phasic dopamine activity encodes reward prediction errors, how this signal drives learning through receptor-level mechanisms remained a question. Specifically, if D1 receptors on the direct pathway facilitate learning from positive outcomes and D2 receptors on the indirect pathway facilitate learning from negative outcomes, then manipulating dopamine levels should selectively affect positive and negative feedback learning in opposite directions. Frank, Seeberger, and O'Reilly (2004) tested this prediction by examining procedural learning in Parkinson's disease patients on and off dopamine medication, using probabilistic selection and transitive inference tasks. Patients off medication — with depleted dopamine — showed enhanced learning from negative feedback but were impaired at learning from positive outcomes, while patients on medication showed the reverse pattern. These results confirmed the predicted double dissociation, establishing that positive and negative feedback learning are mediated by separable dopaminergic mechanisms through D1 and D2 receptors respectively.

4.2.2. Computational Psychiatry

- Huys et al. 2016 - review of comp. psychiatry
- uses computational models to characterize psychiatric ocnditions mechanistically rather than symptom based
- Theory driven vs. Data driven
- RL parameters as biomarkers

- Q-learning parameters as biomarkers - individual differences in alpha values reflect real biological variation

4.2.3. Q-Learning in Psychitaric Research

- Frank et al. 2007
- Introduce their methods
  - probabilistic selection task
  - split alpha into a+ and a-
  - fitting D1/D2 receptors
  - DNA profiles for receptor variants
  - recovering disorder profiles

4.2.4. Our Simulation

- replication of Frank et al. (2007) logic in maze environment
- four agents with different alpha values
- learning curves and TD error results
- Interpretation; connect back

References